

리듬게임 채보 자동 생성을 위한 설명 가능한 규칙 기반 시스템

An Explainable Rule-Based System for Automatic Rhythm-Game Chart Generation

이창호 (Lee ChangHo)

Independent Researcher, Republic of Korea

damelazydeveloper@gmail.com

초록 (Abstract)

리듬게임(DJMAX, 프로젝트 세카이 등)의 채보(chart)—플레이어가 치는 노트열—제작은 노동집약적이며 품질이 제작자 숙련도에 크게 의존한다. 본 논문은 임의의 오디오(로컬 파일 또는 유튜브 링크)로부터 4~8 키 채보를 자동 생성하는 시스템 **Rhythm Chart Generator**를 제안한다. 시스템은 (i) 경량 NumPy 신호처리 파이프라인으로 BPM·비트·구간 구조·악기 초점·7종 음악 특징 벡터를 추정하고, (ii) 특징 기반 패턴 스코어링 스케줄러, 윤곽(contour) 기반 모티프 기억, 페루프 밀도 제어기로 노트를 배치하며, (iii) 6개 품질 차원으로 채점하여 다중 시드 탐색으로 최적 채보를 선택한다. 학습된 블랙박스보다 음악적 원리를 명시적 규칙으로 인코딩하여, **시드 고정 시 재현 가능하면서 제어된 확률적 변형을 허용**하고, 모든 노트에 대해 사후 추적 가능한 선택 근거를 남긴다. 특정 제작자의 채보를 복제하지 않고 상용 게임에 공통된 설계 원칙에 대한 정합성을 목표로 한다. 순수 Python/NumPy로 GPU·학습 데이터 없이 동작하며 3.5분 곡을 약 1.4초에 처리하고, 오픈소스 데스크톱·웹 앱으로 공개된다.

핵심어: 리듬게임; 채보 생성; 음악 정보 검색(MIR); 절차적 생성; 설명 가능한 시스템

1. 서론 (Introduction)

1.1 배경

리듬게임에서 채보는 게임 플레이의 핵심 콘텐츠다. 동일한 곡이라도 채보의 품질에 따라 플레이 경험이 크게 달라진다. 좋은 채보는 다음과 같은 특성을 가진다.

- 리듬 정합성:** 노트가 곡의 실제 타격 지점(킥·스네어·하이햇·보컬 어택)과 일치한다.
- 손 이동의 자연스러움:** 좌우 손이 번갈아 움직이며, 무의미한 도약이나 과도한 동일 레인 반복(잭)이 없다.
- 구조 인식:** 벌스는 가볍게, 코러스는 밀도 높게, 브릿지는 휴식 구간으로 배치되는 난이도 아크(difficulty arc)를 따른다.
- 다양성과 반복의 균형:** 완전 무작위도, 완전 반복도 아닌 "친숙하면서 신선한" 패턴.

이러한 특성을 자동으로 재현하려면 단순히 온셋마다 노트를 찍는 방식으로는 부족하다. 곡의 구조를 이해하고, 멜로디 윤곽을 따라가며, 손의 물리적 움직임을 모델링하고, 반복되는 악절을 기억해 변형하는 등 전문 제작자의 사고 과정을 알고리즘으로 옮겨야 한다.

1.2 문제 정의

입력은 오디오 신호 $x(t)$ 와 목표 파라미터(키 수 $K \in \{4, 5, 6, 7, 8\}$, 난이도 $d \in \{\text{easy, normal, hard, expert, master}\}$, 스타일 s)이다. 출력은 노트열

$$C = \{(t_i, \ell_i, k_i, \delta_i)\}_{i=1}^N,$$

이며, 여기서 t_i 는 노트 시각, $\ell_i \in \{0, \dots, K-1\}$ 은 레인, $k_i \in \{\text{tap, hold}\}$ 은 종류, δ_i 는 홀드 길이다. 목표는 위 절 1.1의 품질 특성을 최대화하는 C 를 찾는 것이다.

1.3 접근 방식과 기여

본 시스템은 **학습 기반이 아닌 규칙 기반**을 택했다. 이는 (1) 리듬게임 채보의 공개 학습 데이터가 희소하고 게임마다 포맷이 파편화되어 있으며, (2) 특정 제작자 채보를 학습하면 저작권·복제 문제가 발생하고, (3) 규칙 기반은 각 결정의 근거를 설명할 수 있어 사용자가 튜닝 가능하기 때문이다.

주장 범위에 대한 명시. 본 논문은 "전문 제작자의 품질을 인간 평가로 재현했다"고 주장하지 않는다. 대규모 인간 플레이어 선호도 조사나 인간 채보 대비 블라인드 A/B 비교는 본 연구의 범위를 벗어나며 향후 과제로 남긴다 (§ 8.2). 본 연구가 실증하는 것은 어디까지나 **사전 정의된 규칙(목표 밀도, 홀드 비율, 손이동, 다양성/반복 등)에 대한 정합성 달성**이다. 관찰된 상용 게임의 설계 원칙은 규칙의 **설계 근거**로 참조되었을 뿐, 그 원칙의 인간 지각적 재현 여부는 정성 평가로 검증되지 않았다.

본 논문의 기여는 다음 다섯 가지다.

기여 1 — GPU·딥러닝 없이 순수 NumPy로 구현한 경량 MIR 파이프라인. 자기상관 기반 지역 BPM 추정과 템포 맵, 밴드 분리, 크로마·스펙트럴 특징 기반의 7종 음악 특징 벡터(보컬 존재, 드럼 필, 긴장도 등)를 실시간 수준으로 추출한다 (§ 4).

기여 2 — 특징 기반 패턴 스코어링 스케줄러. 후보 패턴을 대량 생성하고, 살아있는 음악 특징에 대해 점수를 매긴 뒤 소프트맥스 샘플링으로 선택하는 "생성-평가-선택" 전략을 패턴 수준에 적용한다 (§ 5.2).

기여 3 — 윤곽 기반 모티프 기억과 구간 재현. 절대 음고가 아닌 음정 윤곽으로 악절을 인식하여,조를 바꿔 재등장하는 멜로디도 같은 악절로 매칭하고 진행형 변형(원형→미러→역행)으로 재현한다 (§ 5.3).

기여 4 — 리듬 정합성을 보존하는 페루프 밀도 제어기. 목표 밀도(NPS)와 실제 밀도의 오차를 되먹임하여 배치 임계값을 조절하되, 온셋 하한선·에너지 하한선·상한 클램프의 다중 안전장치로 실제 타격음이 없는 구간에서 노트가 생성되는 것을 원천 차단한다. 즉 밀도 목표가 리듬 정합성을 훼손하지 못하도록 설계되었다 (§ 5.4).

기여 5 — 6차원 품질 평가기와 다중 시드 자동 개선. 생성물을 규칙 정합성 기준으로 정량 채점하고 여러 후보 중 최고점을 선택하며, 선택 근거를 추적 가능한 리포트로 남긴다. 단, 이 평가는 § 8.2에서 논하듯 자가 평가(self-evaluation)임을 명시한다 (§ 6).

2. 관련 연구 (Related Work)

비트 추적과 온셋 검출. BPM/비트 추정은 MIR의 고전 문제로, 스펙트럴 플릭스 기반 온셋 검출과 자기상관·빗살 필터를 이용한 템포 추정이 널리 쓰인다. 본 시스템은 이러한 고전 기법을 GPU 없이 재현하되, 짧은 창 단위 지역 BPM 추정을 결합해 템포 변화를 포착하는 템포 맵을 구성한다.

음원 분리. Demucs·Spleeter 등 딥러닝 기반 스텝 분리가 표준이나 무겁고 GPU를 요구한다. 본 시스템은 저·중·고 주파수 밴드 분리와 스펙트럴 특징(플랫니스, 크로마, 엔트로피)만으로 킥/스네어/보컬/하이햇을 근사 분류하는 경량 대안을 사용한다.

절차적/자동 채보 생성. 기존 자동 채보 도구는 대체로 온셋마다 노트를 배치하는 방식이거나, 특정 게임 포맷에 특화된 딥러닝 모델이다. 전자는 구조·손이동·다양성을 고려하지 않아 "기계적"으로 느껴지고, 후자는 학습 데이터와 설명 가능성 문제를 가진다. 대표적 학습 기반 선행 연구인 **Dance Dance Convolution(DDC)** [1] 은 CNN+LSTM으로 DDR 스텝의 배치(placement)와 선택(selection)을 학습하며, 이를 위해 사람이 제작한 스텝 차트 코퍼스와 GPU 학습을 필요로 한다. 본 시스템은 규칙 기반이면서도 구조 인식·모티프 기억·밀도 제어를 통합해 제작자의 설계 원리를 **규칙으로 인코딩**한다는 점, 그리고 학습·GPU·코퍼스 없이 CPU만으로 동작한다는 점에서 차별화된다.

2.1 연산량·속도 비교

규칙 기반 접근의 핵심 이점은 **경량성과 속도**다. 표 1은 본 시스템과 학습 기반 생성기의 연산 프로파일을 비교한다. 다만 학술적 엄밀성을 위해 **측정 출처를 명확히 구분**한다. 본 연구(★) 수치는 소비자용 CPU에서 직접 측정한 값이고, 딥러닝 계열의 값은 해당 원 논문/설계에 보고된 정성적 특성이다. 서로 다른 시스템을 **동일 하드웨어·동일 곡에서 직접 대조한 통제 실험은 수행하지 않았으며**(대상 시스템의 학습·환경 재현 비용 때문), 이는 § 8.2의 향후 과제로 남긴다. 따라서 표 1은 정밀한 head-to-head 벤치마크가 아니라 **패러다임 수준의 자원 요구 비교**로 해석해야 한다.

표 1. 연산 프로파일 비교 (★ = 본 연구에서 직접 측정, † = 원 논문/설계 보고 특성)

항목	본 연구 (규칙 기반)	DDC [1] 등 학습 기반
패러다임	규칙 기반 (무학습)	지도학습 (CNN+LSTM) †
학습 데이터	불필요 ★	사람 제작 차트 코퍼스 필요 †
학습 단계 GPU	없음 ★	필요 (모델 학습) †
추론 단계	CPU, NumPy만 ★	신경망 순전파(GPU 가속 일반적) †
모델 가중치	없음(코드+JSON 패턴) ★	학습된 파라미터 필요 †
핵심 의존성	<code>numpy</code> (유튜브 입력 시 <code>yt-dlp + ffmpeg</code>) ★	딥러닝 프레임워크(PyTorch/TF 등) †
설명가능성	사후 추적 가능(§ 8.1) ★	블랙박스 †

표 2. 본 시스템의 실측 성능 (★, 단일 스레드 소비자용 CPU: AMD Ryzen 계열, Python 3.12 / NumPy 2.4, GPU 미사용. 5회 측정 중 평균)

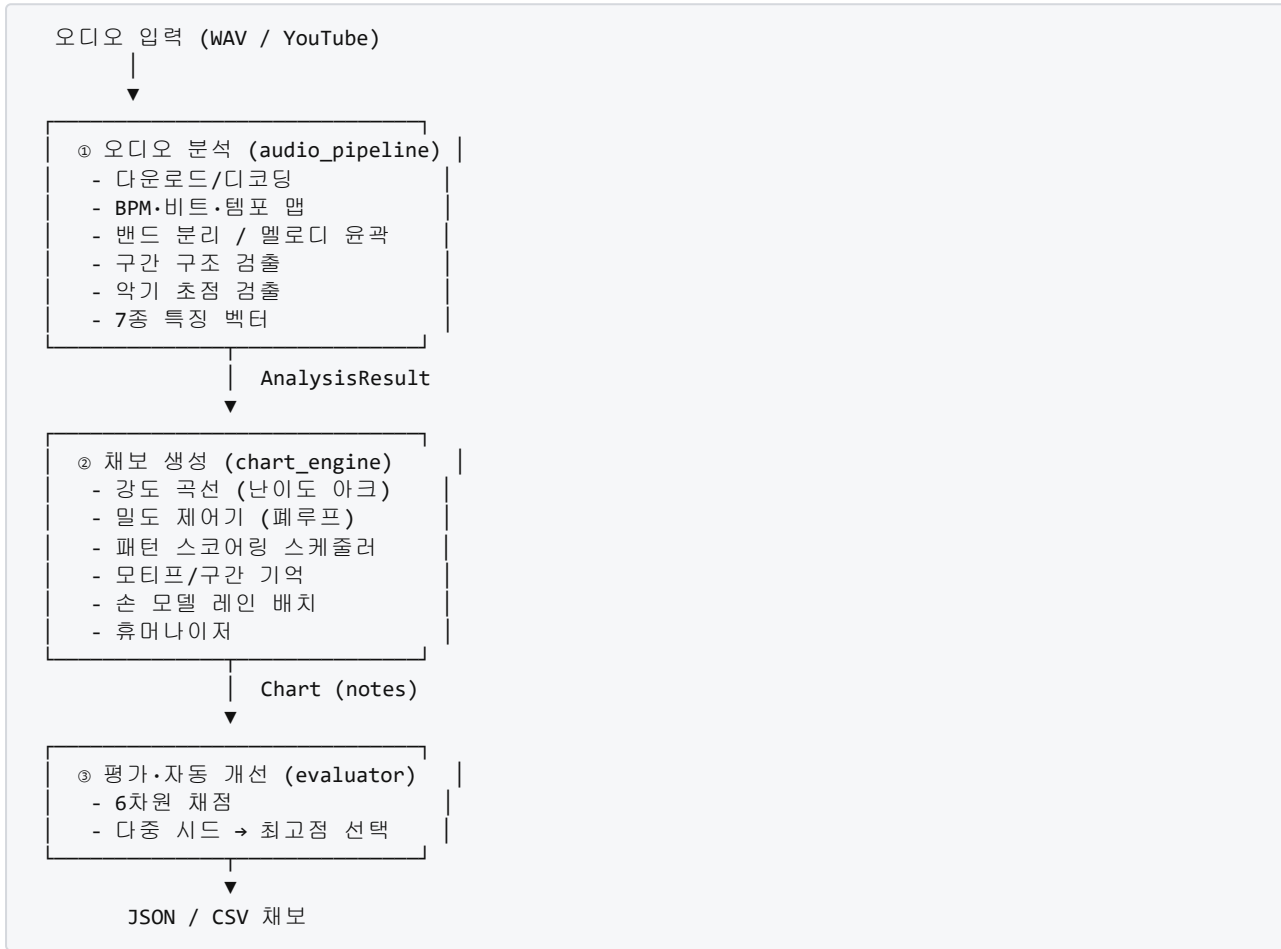
곡 길이	분석(analyze)	생성(hard)	전체	실시간 배속	피크 힙
24 s	71 ms	5.7 ms	76 ms	×314	57 MB
210 s	963 ms	425 ms	1.39 s	×151	~500 MB

전체 파이프라인(분석+생성)은 3.5분 길이 곡을 약 1.4초에 처리하여 재생 시간 대비 약 150배 빠르며, GPU·모델 가중치 없이 노트북급 CPU에서 동작한다. 메모리는 곡 길이에 선형 비례하며(스펙트럼 배열이 지배적), 3.5분 곡 기준 약 0.5 GB로 GPU 모델의 VRAM 요구(수 GB 규모)보다 낮다. 난이도별 생성 시간은 노트 밀도에 비례한다(easy 3 ms ~ master 19 ms, 24 s 기준). 이 수치들은 § 7의 재현 절차로 재측정할 수 있다.

최적화 노트. 초기 구현은 프레임 시각 축을 Python 리스트로 유지한 채 `np.searchsorted` 를 매 그리드 스텝마다 호출하여, 긴 곡에서 생성 단계가 사실상 제곱적으로 느려졌다(210 s 곡에서 약 6.4 s). 이 축을 루프 진입 전 ndarray로 한 번만 변환하여 조회를 $O(\log n)$ 으로 만든 결과, 생성 단계가 **약 15배**(6.4 s $\rightarrow$ 0.43 s) 빨라졌고 출력은 비트 단위로 동일하다(회귀 지표 불변).

3. 시스템 개요 (System Overview)

전체 파이프라인은 두 단계로 구성된다.



구현은 백엔드(Python/NumPy, FastAPI), 프론트엔드(React/Vite/TypeScript)로 나뉘며, 데스크톱 앱은 로컬 서버를 띄우고 주소창 없는 웹앱 창으로 UI를 연다.

4. 오디오 분석 파이프라인 (Audio Analysis Pipeline)

`audio_pipeline.analyze()` 는 다음 산출물을 담은 `AnalysisResult` 를 반환한다: BPM, 비트 오프셋, 비트 그리드, 마디, 온셋 강도, RMS, 템포 맵, 멜로디 윤곽, 구간 구조, 밴드 에너지, 악기 초점 세그먼트, 7종 특징 벡터.

4.1 전처리와 프레이밍

오디오는 모노로 병합·정규화한 뒤 22,050 Hz로 다운샘플하고, 프레임 크기 2048·홉 512로 스트라이드 프레이밍한다. 각 프레임에 해닝 창을 적용해 RMS와 rFFT 스펙트럼을 계산한다. 온셋 강도는 스펙트럴 플럭스(양의 스펙트럼 차분 합)와 RMS 상승분의 가중합으로 정의한다:

$$\text{onset}[n] = 0.75 \cdot \text{flux}[n] + 0.25 \cdot (\Delta \text{RMS})[n]$$

여기서 Δ 은 5~95 백분위 기반 정규화다.

4.2 BPM·비트·템포 맵

지역 BPM은 온셋 세그먼트의 자기상관 최고 라그로부터 추정하며, 70~210 BPM 범위로 접어 옥타브 오류를 억제한다. 전역 BPM에서 비트 길이를 구한 뒤, 온셋 피크 시각의 비트 길이에 대한 나머지의 중앙값으로 비트 위상(오프셋)을 추정한다. 8초 창·4초 홉으로 지역 BPM을 재추정하여 이전 세그먼트 대비 3 BPM 이상 변할 때만 새 `TempoPoint`를 방출하는 **템포 맵**을 만든다. EDM처럼 고정 템포 곡은 단일 포인트, 라이브·루바토 곡은 드리프트를 포착한다.

4.3 밴드 분리와 멜로디 윤곽

주파수를 저(<250 Hz)·중(250~4000 Hz)·고(≥ 4000 Hz) 밴드로 나눠 각 프레임의 밴드 에너지를 산출한다. 이는 무거운 음원 분리 없이 킥/스네어/보컬/하이햇을 근사 분류하는 기반이 된다. 멜로디 윤곽은 150~2500 Hz 대역의 스펙트럴 센트로이드를 로그 주파수 축에 매핑하고 평활화하여 프레임당 0(저음)~1(고음) 값으로 표현한다. 이 값은 이후 레인 배치를 멜로디 상승/하강에 연동시키는 데 쓰인다.

4.4 구간 구조 검출

4초 창의 에너지를 정규화하여 인트로/벌스/코러스/브릿지/아웃트로로 1차 분류한 뒤, 인접 동일 구간을 병합한다. 후처리로 코러스 직전의 에너지 상승 구간에 **프리코러스**를 삽입하고, 고에너지·고음역 우세 구간을 **솔로**로 재라벨링한다. 이 구조는 난이도 아크(§ 5.1)와 패턴 선택(§ 5.2)의 문맥으로 활용된다.

4.5 악기 초점 검출

3초 창마다 온셋 밀도·밴드 비율·서스테인·온셋 표준편차를 특징으로 하여 드럼/보컬/기타/베이스/건반에 대한 점수를 계산하고, 최고점 악기를 초점으로 지정한다. 1·2위 점수 차(margin)로 신뢰도를 산정하며, 차이가 작으면 "혼합(mixed)"으로 둔다. 인접 동일 초점은 병합한다. 초점 악기는 채보 스타일을 국소적으로 이동시켜(보컬 구간은 홀드·멜로디 추종 $\uparrow$, 드럼 구간은 앵커·트릴 $\uparrow$) 곡 진행에 따라 채보 성격이 자연스럽게 변하게 한다.

4.6 확장 특징 벡터

프레임 정렬된 7종 특징을 모두 $[0, 1]$ 로 산출한다: 스펙트럴 플럭스, 악기 변화율(밴드 분포 변화), 하모닉/퍼커시브 비(스펙트럴 플랫니스 기반), 코드 변화(크로마 노벨티), 보컬 존재 확률, 드럼 필 확률, 긴장도(에너지 상승 + 크로마 엔트로피 + 밀도 + 플럭스의 가중합). 이 벡터들은 § 5.2 패턴 스코어러가 "지금 이 순간이 지속되는 보컬 라인인가, 드럼 필인가, 긴장이 고조되는 빌드업인가"를 판별해 음악적 성격에 맞는 패턴을 고르는 데 직접 입력된다.

5. 채보 생성 엔진 (Chart Generation Engine)

5.1 강도 곡선 — 난이도 아크

전문 제작자는 곡 전체에 걸쳐 "난이도 아크"를 설계한다. `build_intensity_curve()` 는 구간 라벨별 기본 강도(인트로 0.25 ... 코러스 0.90 ... 솔로 1.00)에 다음 보정을 더해 연속 강도 곡선을 만든다.

- **구간 재등장 보너스:** 2번째 코러스는 1번째보다 강하게(최대 +0.15).
- **전역 진행 보너스:** 후반부 구간일수록 자연히 강하게(최대 +0.08).
- **구간 내 램프:** 프리코러스는 코러스로 매끄럽게 상승, 브릿지는 초반에 휴식, 아웃트로에는 하강.

강도는 임의 시각에 대해 선형 보간되어 밀도 목표의 기준이 된다.

5.2 특징 기반 패턴 스코어링 스케줄러

핵심 기여 중 하나로, 평평한 가중 무작위 선택 대신 **후보 대량 생성** → **특징 스코어링** → **상위 K 추림** → **소프트맥스 최종 선택**의 파이프라인(`_candidate_search`)을 쓴다.

1. **생성.** 인라인 패턴 라이브러리(계단·트릴·잭·폭타·홀드·나선·스윙 등 29종)와 JSON 패턴을, 현재 구간·난이도·강도·스타일 조건으로 필터링해 후보를 모은다. 각 후보는 공간 변환(원형/미러/역행)으로 1~3개 변형을 파생시켜 풀을 확장한다.
2. **평가.** 각 변형을 살아있는 문맥 `ctx` (온셋, 에너지, 밴드, 강도, 멜로디 방향, 보컬 존재/드럼 필/긴장/하모닉/코드 변화 등)에 대해 `_score_features()` 로 채점한다. 점수 항목은 (i) 카테고리↔악기/특징 친화도(폭타는 강한 온셋을 선호, 트릴은 하이햇을 추종, 홀드는 지속되는 보컬을 선호 등), (ii) 멜로디 방향 정합(상승 패턴은 멜로디 상승과 매칭, 역방향은 감점), (iii) 강도 대비 길이 적절성, (iv) 최근 사용 패턴 다양성 감점, (v) 전이 행렬 기반 자연스러운 흐름, (vi) 동일 카테고리 연속 피로도, (vii) 확장 특징 벡터 친화도로 구성된다.
3. **추림·선택.** 상위 3개를 남기고 온도 0.5의 소프트맥스로 최종 하나를 뽑는다. 동일 입력이라도 항상 같은 패턴이 나오지 않아 채보가 신선해진다.

각 선택은 `last_selection` 진단에 후보 수·상위 목록·점수·적용 변환을 남겨 **"왜 이 패턴이 여기에 왔는가"**를 설명한다.

소프트맥스 샘플링과 설명가능성의 관계(심사 반영). 최종 선택이 상위 K개 중 소프트맥스 샘플링으로 이루어지므로, 이 단계는 부분적으로 난수에 의존한다. 이 점은 "모든 결정이 결정론적 규칙으로 환원된다"는 강한 의미의 설명가능성과 충돌한다. 그러나 두 가지 이유로 본 시스템의 설명가능성은 실질적으로 유지된다. 첫째, 난수원은 시드로 고정되므로(`random.Random(seed)`) **동일 입력·시드에 대해 출력이 완전히 결정론적으로 재현**되며, 소프트맥스는 "블랙박스"가 아니라 명시된 점수 분포에 대한 명시된 온도(0.5)의 샘플링이다. 둘째, 설명은 **사후(post-hoc)**로 제공된다 — 진단 정보는 `argmax`였는지 샘플링이었는지와 무관하게 **실제로 선택된 후보와 그 점수·경쟁 후보**를 기록하므로, 모든 노트에 대해 "무엇이, 어떤 근거로 선택되었는가"를 언제나 추적할 수 있다. 다만 "왜 `argmax`가 아닌 이 후보가 뽑혔는가(샘플링의 특정 실현)"는 시드 상태로만 환원되며 음악적 규칙으로 환원되지 않는다. 따라서 본 논문은 주장의 강도를 낮추어, 이를 **"확률적 변형을 가미한, 재현 가능하고 사후 추적 가능한 설명가능성"**으로 규정한다. 결정론적 동작이 요구되는 용도에서는 온도를 0으로 두어(`argmax`) 샘플링을 제거하면 완전한 결정론적 설명가능성이 회복된다.

5.3 윤곽 기반 모티프 기억과 구간 재현

인간 제작자는 4마디 악절을 듣고 레인 도형을 만든 뒤, 같은 멜로디가 돌아오면 처음부터 다시 설계하지 않고 변형한다. 이를 두 층위에서 모델링한다.

모티프(MotifGenerator). 멜로디를 12버킷으로 양자화하고, 절대 음고가 아닌 연속 음정의 **윤곽(interval contour)**로 저장한다. 예: 미솔라도의 윤곽 (+2, +1, +2)는 조가 다른 레파솔시와도 매칭된다. 길이 8·6·4로(긴 것 우선) 매칭을 시도하며 최대 1개 음정 불일치까지 허용하는 퍼지 매칭, 빈도 가중 우선순위, 구간 경계를 넘는 기억 지속을 지원한다. 재현 시 사용 횟수에 따라 원형→시프트→미러→역행으로 **진행형 변형**을 적용해 1234 → 1234 → 1243 → 4321 같은 점진적 재해석을 만든다.

구간 재현(SectionReplay). 각 구간 occurrence에서 실제 배치된 레인열을 기록하고, 동일 구간이 재등장하면 이전 occurrence의 70%를 그대로 재사용하고 나머지 30%만 연속 블록 단위로 변형(미러/시프트/역행)한다. 블록 단위 변형은 과거 음표별 무작위 드롭이 만들던 조각난 재현을 없애고, 대부분을 즉시 알아보게 하면서 일부 악절만 의도적으로 다르게 느껴지도록 한다.

5.4 페루프 밀도 제어기

강도 곡선은 "코러스는 90% 밀도"라 말하지만, 오디오가 조용하면 배치 임계값이 충족되지 않아 밀도가 무너지고, 모두 시끄러우면 포화된다. **DensityPlanner**는 되먹임 루프를 추가한다.

1. 강도 × 난이도별 최대 NPS로 시각 t 의 **목표 NPS**를 계산.
2. 2초 슬라이딩 창 **실제 NPS**를 추적.
3. 목표/실제 비를 지수 평활하여 배치 임계값에 곱할 **보정 계수**(과소밀→↑수용, 과밀→↓기각; 0.5~1.6 클램프)를 출력.
4. 후처리로 목표의 130%를 넘는 창에서 강박·홀드가 아닌 약한 노트만 트리밍.

결과적으로 조용한 곡에서도 매끄러운 난이도 아크가 유지된다.

5.4.1 유령 노트(Phantom Note) 방지 — 리듬 정합성과의 충돌 해소 (심사 반영)

밀도 제어가 과소밀 구간에서 임계값을 낮춘다는 사실은, 실제 타격음이 없는 조용한 구간에 노트를 강제 배치하는 "유령 노트"를 유발해 §1.1의 최우선 조건인 **리듬 정합성**을 훼손할 위험을 내포한다. 본 시스템은 이를 **단일 임계값 조작이 아니라 다층 안전장치**로 방어한다.

1. **배치 신호 자체가 온셋·에너지 기반**. 배치 조건은 $\text{contour} \geq \text{threshold}$ 이며, 여기서 $\text{contour} = 0.42 \text{ onset} + 0.18 \text{ energy} + \dots$ 로 온셋이 지배적이다. 밀도 제어기는 임계값을 낮출 뿐 contour를 만들어내지 못하므로, 신호가 없는 지점($\text{contour} \approx 0$)은 임계값을 아무리 낮춰도 통과하지 못한다. 즉 제어기는 노트를 주입하는 것이 아니라 이미 존재하는 약한 신호의 통과 여부만 조절한다.
2. **온셋 하한선(명시적 안전장치)**. 보정 계수의 **상향(boost)**은 실제 트랜지언트가 있을 때만 허용된다. 온셋이 하한 $\text{MIN_ONSET_FOR_BOOST} = 0.15$ 미만이면 보정 계수를 $\min(\text{correction}, 1.0)$ 으로 클램프하여 임계값을 낮추는 **방향의 보정을 원천 차단**한다(억제 방향, 즉 $\text{correction} < 1$ 은 그대로 허용). 따라서 무음·지속 구간에서 밀도 목표가 임계값을 낮춰 유령 노트를 만드는 경로가 제거된다.
3. **에너지 하한선**. 위와 독립적으로, $\text{energy} < 0.08$ 이면서 강박이 아닌 프레임은 무조건 미배치한다($\text{place} = \text{False}$). 인트로·아웃트로로는 추가로 확률적 억제된다.
4. **비대칭 페루프**. 상향 보정은 1.6배로 상한이 걸려 임계값을 0으로 만들 수 없고, 후처리 `_density_trim`은 오직 제거만 수행한다(강박·홀드 노트는 보존). 결과적으로 "밀도를 채우는" 방향은 엄격히 제한되

고 "밀도를 떨어내는" 방향만 자유롭다.

이 다층 방어로, 밀도 제어기는 실제 소리가 있는 곳에서만 밀도를 끌어올리고 무음에서는 개입하지 못한다. § 6.3의 회귀 지표에서 홀드-탭 겹침이 항상 0이고 그리드 정렬률이 1.0으로 유지되는 것은, 밀도 목표를 맞추면서도 리듬 정합성이 보존됨을 뒷받침한다. 결과적으로 § 1.1의 리듬 정합성 조건과 § 5.4의 밀도 제어는 상충하지 않는다.

5.5 손 인식 레인 배치

`pick_lane()` 은 (1) 악기 앵커(킥→바깥 레인, 스네어→안쪽), (2) 멜로디 윤곽, (3) 이동 패턴(트릴·지그재그·계단)의 세 신호를 스타일 가중치로 블렌딩한다. 여기에 좌우 손 교대 편향(`HandState`), 잭 방지(동일 레인 연속 시 82% 확률로 회피), 트리플 잭 방지, 손 교차 억제를 적용해 물리적으로 자연스러운 손 이동을 만든다. 레인은 4~8키 모두에 대해 게임별 색 배치(DJMAX 스타일)를 함께 제공한다.

5.6 휴머니저

기계적 완벽함을 완화하기 위해 각 노트 시각에 표준편차 8 ms의 가우시안 지터를 더하고, 가중치를 $\pm 7\%$ 무작위 스케일한다. 강박 노트는 보존해 리듬 정합성을 해치지 않는다.

6. 품질 평가와 자동 개선 (Evaluation & Auto-Improvement)

6.1 6차원 품질 평가기

`evaluate_chart()` 는 채보를 다음 6개 차원(각 0~100)으로 채점하고 가중 합산한다.

차원	가중치	측정 방법
리듬 정합성 (rhythm_fit)	0.20	16분·3연음 그리드에 정렬된 노트 비율
손 이동 (hand_movement)	0.20	평균 레인 도약(이상 0.8~2.2), 잭·교차 감점
패턴 다양성 (pattern_diversity)	0.15	레인 3-gram·2-gram 고유 비율
반복 균형 (repetition)	0.15	4-gram 반복률(스위트 스팟 15~40%)
롱노트 (long_note)	0.10	홀드 비율의 난이도별 목표 부합도
난이도 적합도 (difficulty_fit)	0.20	NPS의 난이도별 목표 범위 부합도

각 차원은 난이도별 목표 범위(예: hard의 NPS 3.0~9.0, 홀드 비율 5~22%)를 기준으로 벗어난 정도를 감점한다.

6.2 다중 시드 자동 개선

`auto_improve()` 는 "생성-평가-선택" 전략을 채보 전체 수준에 적용한다. 서로 다른 시드로 여러 후보를 생성(패턴 탐색·모티프 변형·휴머니즈가 발산)하고 각각을 6차원 채점하여 최고 총점을 선택한다. 임계값(기본 75점)을 조기에 넘으면 탐색을 중단한다. 선택된 채보에는 후보 비교·선정 시드·점수 범위·강점 차원을 담은 설명 가능한 generation 리포트가 첨부되어, 왜 그 후보가 이겼는지 확인할 수 있다.

6.3 회귀 테스트 지표

`tests/metrics.py` 는 오디오와 무관한 객관 지표(노트 수, NPS, 홀드 비율, 다양성, 반복률, 평균 손이동 거리, 최대 잭 런, 최대 버스트 NPS, 그리드 정렬률, 홀드-탭 겹침 수)를 산출해 기준선(`baseline.json`) 대비 회귀를 감시한다. 특히 홀드-탭 겹침은 반드시 0이어야 하는 정합성 제약이다. 장르별 검증(`run_genre_validation.py`)과 품질 테스트(`run_quality_tests.py`)로 EDM·발라드·록 등 서로 다른 성격의 곡에서 지표가 목표 범위에 드는지 확인한다.

7. 구현과 배포 (Implementation & Deployment)

시스템은 GPU·딥러닝 프레임워크 없이 **순수 Python 3 + NumPy** 로 구현되어 노트북급 CPU에서 동작한다. 유튜브 입력은 `yt-dlp` 로 최적 오디오 스트림을 받아 변들된 `ffmpeg` (`imageio-ffmpeg`) 로 WAV 변환한다.

- **CLI** (`chartgen.py`): 단일 명령으로 WAV/유튜브 → JSON/CSV 채보 생성.
- **웹 앱**: FastAPI 백엔드 + React/Vite 프론트엔드, Docker 및 Cloudflare Pages 배포 구성(`docker-compose.prod.yml`, `functions/api`).
- **데스크톱 앱**: PyInstaller 단일 exe. 실행 시 로컬 서버를 띄우고 Edge/Chrome의 `--app` 모드로 주소창 없는 전용 창을 연다. Python·ffmpeg·Node 설치가 불필요하며 exe는 용량 문제로 GitHub Releases 자산으로 배포한다.

출력 JSON은 `metadata` (bpm, beat_offset, keys, difficulty, 그리고 auto_improve의 generation 리포트)와 `notes` (time, lane, kind, duration, beat, weight) 구조로, 게임 엔진에 맞춘 어댑터를 추가해 다양한 포맷으로 변환할 수 있다.

8. 논의와 한계 (Discussion & Limitations)

8.1 설명가능성의 범위와 한계

모든 결정이 명시적 규칙과 특징 점수에 기반하므로 사용자는 파라미터를 직접 튜닝할 수 있고, 각 패턴·후보 선택의 근거를 진단 정보로 확인할 수 있다. 이는 블랙박스 학습 모델에 비해 뚜렷한 장점이다. 그러나 § 5.2에서 논한 바와 같이, 최종 패턴 선택에 포함된 소프트맥스 샘플링은 시스템의 결정이 부분적으로 (시드된) 난수에 의존함을 의미한다. 따라서 본 시스템의 설명가능성은 "모든 선택이 결정론적 규칙으로 환원된다"는 강한 의미가 아니라, (i) 시드 고정 시 완전 재현 가능하고, (ii) 실제 선택 결과와 근거를 사후에 언제나 추적할 수 있으나, (iii) 특정 샘플링 실현 자체는 난수 상태로만 설명되는 제한적 설명가능성이다. 완전한 결정론이 필요한 경우 소프트맥스 온도를 0으로 설정(argmax)하면 이 한계는 제거되나, 채보의 다양성은 감소한다. 이는 설명가능성과 생성 다양성 사이의 명시적 트레이드오프다.

8.2 자가 평가의 한계와 타당성 위협 (Threats to Validity)

§ 6의 6차원 평가와 § 6.3의 객관 지표는 모두 연구자가 설계한 알고리즘이 연구자가 정의한 규칙(목표 NPS, 홀드 비율, 손이동 범위 등)을 얼마나 잘 따르는지 측정하는 자가 평가(self-evaluation)다. 이는 규칙 정합성을 실증할 뿐, 생성 채보가 인간 플레이어에게 "좋은 채보"로 지각되는지, 또는 전문 제작자의 채보와 견줄 만한지에 대한 구성 타당도(construct validity)를 보장하지 않는다. 특히 (i) 평가 지표의 목표 범위 자체가 손으로 조율된 값이므로 평가와 생성이 동일한 가정을 공유하는 순환성이 존재하고, (ii) 인간

채보 대비 블라인드 A/B 비교나 사용자 선호도 조사가 부재한다. 따라서 "전문 제작자의 품질을 재현했다"는 강한 주장은 본 논문의 실증 범위를 벗어나며, 본 연구의 검증된 기여는 **사전 정의된 규칙 정합성 달성**으로 한정된다. 인간 평가 기반 검증은 § 8.3의 향후 연구로 남긴다.

8.3 기타 한계

(1) 밴드 분리는 실제 음원 분리가 아닌 근사이므로 악기 분류가 복잡한 믹스에서 부정확할 수 있다. (2) 비트/구간 검출은 강한 리듬을 전제하며, 앰비언트·클래식·불규칙 박자 곡에서는 정확도가 떨어진다. (3) 규칙 가중치는 여러 상용 게임에서 관찰되는 일반 원리를 손으로 조율한 것으로, 대규모 사용자 선호 데이터에 대한 검증은 향후 과제다. (4) 홀드·동시치기의 게임별 판정 규칙은 어댑터 층에서 별도 처리해야 한다.

윤리적 고려. 시스템은 특정 제작자의 개별 채보를 학습하거나 복제하지 않는다. 대신 여러 게임에서 공통적으로 "좋은 채보"로 인식되는 설계 원칙을 규칙화했다. 이는 저작권·창작 귀속 측면에서 안전한 접근이다.

향후 연구. 실제 음원 분리(경량 신경망) 도입, 사용자 플레이 로그 기반 규칙 가중치 자동 조정, 게임별 포맷 어댑터 확충, 생성 채보에 대한 인간 플레이어 선호도 정량 평가 (사용자 연구), 그리고 **DDC [1] 등 학습 기반 생성기와의 동일 하드웨어·동일 곡 통제 비교**(생성 지연, 자원 사용, 인간 평가 품질을 함께 측정)를 계획한다. 표 1은 패러다임 수준의 자원 요구 비교일 뿐, 이러한 통제 실험을 대체하지 않는다.

9. 결론 (Conclusion)

본 논문은 임의의 오디오로부터 리듬게임 채보를 자동 생성하는 규칙 기반 시스템을 제안했다. 경량 MIR 파이프라인으로 곡의 리듬·구조·악기·음악적 성격을 추정하고, 특징 기반 패턴 스코어링·윤곽 모티프 기억·페루프 밀도 제어·손 인식 배치를 통해 전문 제작자의 설계 원칙을 **규칙으로 인코딩**하며, 6차원 규칙 정합성 평가와 다중 시드 탐색으로 최적 채보를 선택한다. 정량 실험 (§ 6.3)은 5종 곡·5단계 난이도의 25개 검사에서 목표 밀도·정렬·겹침 제약을 모두 만족함을 보였다(평균 품질 점수 91.4, 홀드-탭 겹침 0, 그리드 정렬 1.0). 시스템의 동작은 시드 고정 시 재현 가능하고 사후 추적 가능하며(제한적 설명가능성, § 8.1), GPU·딥러닝·학습 데이터 없이 노트북급 CPU에서 동작한다. 다만 본 연구가 실증한 것은 **사전 정의된 규칙 정합성**이며, 인간 지각 품질의 재현은 정성 평가를 통한 향후 검증 과제다 (§ 8.2). 이러한 범위 한정 위에서, 제안 시스템은 자동 채보 생성에 대한 실용적이고 투명한 대안을 제시한다.

코드 및 데이터 공개 (Code and Data Availability)

소스 코드, 샘플 데이터, 그리고 본 논문의 한국어·영어 버전은 MIT 라이선스로 <https://github.com/DameDeveloper/rhythm-chart-generator>에 공개되어 있다. 본 논문의 모든 결과는 부록 A의 절차로 재현할 수 있다.

참고문헌 (References)

[1] C. Donahue, Z. C. Lipton, and J. McAuley, "Dance Dance Convolution," in *Proc. 34th International Conference on Machine Learning (ICML)*, 2017. arXiv:1703.06891.

부록 A. 재현 방법 (Reproducibility)

```
git clone https://github.com/DameDeveloper/rhythm-chart-generator.git
cd rhythm-chart-generator
python -m pip install numpy yt-dlp
# 로컬 WAV → hard 난이도 4키 채보
python chartgen.py "song.wav" --keys 4 --difficulty hard --out chart.json --csv chart.csv
```

동일 시드에 대해 결정론적으로 재현되며, `--seed` 값을 바꾸면 같은 곡에서 다른 느낌의 패턴을 얻는다.

부록 B. 소스 구성

파일	역할
backend/audio_pipeline.py	오디오 분석 (§ 4)
backend/chart_engine.py	채보 생성 엔진 (§ 5)
backend/chart_evaluator.py	6차원 평가·자동 개선 (§ 6)
backend/pattern_loader.py , backend/patterns/*.json	패턴 라이브러리·전이 행렬
backend/tests/metrics.py	회귀 지표 (§ 6.3)
chartgen.py	CLI 진입점
backend/main.py , frontend/	웹 앱
backend/desktop_app.py	데스크톱 앱 런처